SQL QUERIES SCREENSHOT

1. SELECT *

```
58 •  Select * from Customers;
59
```

Result Grid | Filter Rows: | Edit:

customer_id	customer_name	city
1	Pujan Patel	Morbi
2	Khush Patel	Kutch
3	Deep Patel	Surat
4	Ramya Patel	Vapi
NULL	NULL	NULL

2. SELECT + WHERE

Explanation: Retrieves customer records from the Customers table where the city is Morbi.

```
60 •  select * from Customers where city = 'Morbi';
```

Result Grid | Filter Rows: | Edit: | Export/Import:

customer_id	customer_name	city
1	Pujan Patel	Morbi
NULL	NULL	NULL

3. ORDER BY

Explanation: Sorts all orders in descending order based on the total order amount.

```
62 •  SELECT * from Orders
63    ORDER BY total_amount DESC;
```

Result Grid | Filter Rows: | Edit:

order_id	customer_id	order_date	total_amount
103	1	2024-02-01	320.00
101	1	2024-01-10	250.00
102	2	2024-01-15	180.00
104	3	2024-02-05	150.00
NULL	NULL	NULL	NULL

4. GROUP BY + SUM

Explanation: Groups orders by customer and calculates the total sales for each customer.

```
65 •  SELECT customer_id, SUM(total_amount) AS total_sales
66    FROM Orders
67    GROUP BY customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	total_sales
1	570.00
2	180.00
3	150.00

5. AVG Aggregate Function

Explanation: Calculates the average value of all orders in the Orders table.

```
69 •  SELECT AVG(total_amount) AS avg_order_value
70    FROM Orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

avg_order_value
225.000000

6. INNER JOIN

Explanation: Combines Customers and Orders tables to display matching customer and order information.

```
72 •  SELECT o.order_id,c.customer_name,o.total_amount
73    FROM Orders o
74    INNER JOIN Customers c
75    ON o.customer_id = c.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cont

order_id	customer_name	total_amount
101	Pujan Patel	250.00
102	Khush Patel	180.00
103	Pujan Patel	320.00
104	Deep Patel	150.00

7. LEFT JOIN

Explanation: Displays all customers and their orders, including customers who may not have placed any orders.

```
77 •  SELECT c.customer_name,o.order_id
78    FROM Customers c
79    LEFT JOIN Orders o
80    ON c.customer_id = o.customer_id;
```

esult Grid | Filter Rows: | Export:

customer_name	order_id
Pujan Patel	101
Pujan Patel	103
Khush Patel	102
Deep Patel	104
Ramya Patel	NULL

8. RIGHT JOIN

Explanation: Displays all orders and their corresponding customers, including orders without matching customer records.

```
82 •  SELECT c.customer_name,o.order_id
83    FROM Customers c
84    RIGHT JOIN Orders o
85    ON c.customer_id = o.customer_id;
```

esult Grid | Filter Rows: | Export:

customer_name	order_id
Pujan Patel	101
Pujan Patel	103
Khush Patel	102
Deep Patel	104

9. Subquery

Explanation: Retrieves orders whose amount is greater than the average order amount.

```
88 •  SELECT customer_id,total_amount
89    FROM Orders
90    WHERE total_amount >
91    (
92        SELECT AVG(total_amount)
```

Result Grid | Filter Rows: | Export:

customer_id	total_amount
1	250.00
1	320.00

10. CREATE INDEX

Explanation: Creates an index on the customer_id column to improve query performance.

```
107 •  SELECT *
108    FROM Orders
109    WHERE customer_id = 1;
```

Result Grid | Filter Rows: | Edit:

order_id	customer_id	order_date	total_amount
101	1	2024-01-10	250.00
103	1	2024-02-01	320.00
NULL	NULL	NULL	NULL